WOXSEN UNIVERSITY / PBL 10 / WEB TECHNOLOGIES

Inclusive Smart Form Builder

An accessibility-first form builder in vanilla JavaScript. It renders a live form from a field configuration and validates it with errors a screen reader can actually follow.

Mohammed Amaan Khan
25WU0101075 / CSE Rhinos / II Year B.Tech

Lighthouse	axe violations
100	0

The problem

Forms are where inaccessible websites fail hardest, and the failure is never a styling problem.

Unusable, not awkward

An unlabelled input gives a screen reader nothing to announce. An error shown only as a red border is invisible to anyone who cannot see it.

No longer optional

The European Accessibility Act applies from June 2025, and India's RPwD Act already requires digital services to be accessible.

The real question

Not whether JavaScript can create an input. What the generated markup has to contain for the form to work for everyone.

How it works

One array of plain field objects is the single source of truth. Every change notifies the preview, the JSON, the HTML export and the accessibility checks, so none of them can drift out of step.

`schema.js`	Field types and validation rules
`store.js`	The one array, and who to notify
`renderer.js`	Builds the DOM, labels and ARIA
`validator.js`	The rules and when they run
`audit.js`	Checks the DOM it actually built

WHAT IT GENERATES FOR ONE REQUIRED EMAIL FIELD

```
<label for="email">Email address</label>
<p id="email-hint">We send the link here.</p>
<input id="email" type="email"
  autocomplete="email"
  aria-required="true"
  aria-invalid="false"
  aria-describedby="email-hint email-error">
<p id="email-error"></p>
```

The error element is rendered empty and stays. Nothing is ever built from an HTML string, so a label someone types can only ever become text.

Four decisions that did the work

Each had an easier alternative

01

The error element is always there, and empty

Creating it only on failure is unreliable, because some screen readers will not re-read a description added after the field was last focused. An empty element adds nothing to the description, so the link can be made once and never rewired.

02

aria-invalid is the only source of truth

The red border is a CSS rule reading that attribute, not a class. Because the styling and the assistive technology read the same thing, a field cannot be made to look invalid without actually being invalid.

03

Silent on blur, spoken on correction

Announcing an error as focus leaves interrupts the user in their next field. A failed submit focuses an error summary instead, which both announces the problem and links to every field involved.

04

No native maxlength, and no browser bubbles

Native maxlength silently refuses further typing. It explains nothing to anyone, and nothing at all to a screen reader user, who just finds the keyboard has stopped working. The limit is checked in JavaScript so the message can say what is wrong.

What the testing found

Measured, not asserted

100

Lighthouse accessibility, under both mobile and desktop emulation. 30 audits passed, none failed.

0

axe-core violations across five interface states, including after a failed submit.

144

Automated assertions across three suites, all passing on every change.

27

Responsive measurements. Nine widths from 320px, in three display states each.

Four real defects this testing found, and fixed

Radio groups were never marked invalid. The attribute went on the inputs, not the group that carries the state.

Label in name, WCAG 2.5.3. An aria-label replaced the visible text instead of extending it, so speech input could not address the button.

A short phone number got the wrong message. One pattern was doing two jobs, so a length problem read as a format problem.

A name that was silently dropped. aria-labelledby is prohibited on a plain div, so the error summary heading never reached its accessible name.

A score of 100 means no detectable failures, not that a page is accessible

Where it lives

Source code	`github.com/amaancoderx/inclusive-smart-form-builder`
Live demo	`inclusive-smart-form-builder--amaanbuilds.replit.app`
Replit project	`replit.com/@amaanbuilds/inclusive-smart-form-builder`

`docs/REPORT.md`

Problem, architecture, and every design decision with the reasoning behind it

`docs/ACCESSIBILITY-AUDIT.md`

Lighthouse and axe results, keyboard walkthrough, responsive matrix, WCAG mapping

`tests/`

Four suites you can run with npm test, plus the saved Lighthouse report

Mohammed Amaan Khan

`25WU0101075 / CSE Rhinos / II Year B.Tech`

Woxsen University, School of Technology
PBL 10, Web Technologies

HTML5, CSS3 and vanilla JavaScript. No frameworks, no build step, no runtime dependencies.